FACULTY OF ENGINEERING & TECHNOLOGY

ELECTRICAL AND COMPUTER ENGINEERING DEPARTMENT

ADVANCED DIGITAL DESIGN

ENCS3310

PROJECT

Prepared by:

Student Name: Khaled Hammad **Student NO.** 1220857

Instructor: Abdallatif Abuissa

Section:4

Date: 26/12/2024

Contents

Table of Figures

Brief Introduction

In this project, we aim to design a structural comparator for both signed and unsigned numbers to see how comparisons between two numbers are performed logically. The design will show how manual handling of logic gates is achieved using the Active-HDL tool and Verilog programming. This project will include a complete code for verification to ensure the functionality of the comparator. We will evaluate the design by introducing errors to do verification that will discover the error and write it to the console screen.

Brief theoretical overview

What is a Comparator?

A digital combinational circuit used to compare the magnitude of two binary numbers to determine the equality or non-equality is called a comparator. Therefore, the main function of a comparator is to compare the values of input numbers and produce an output indicating whether the numbers are equal or specifies which of the numbers is greater [1].

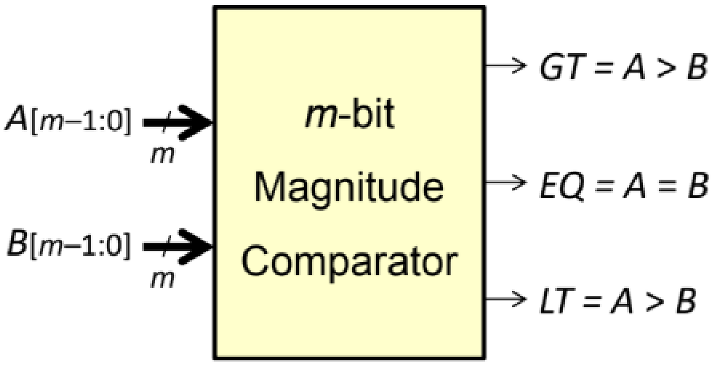

Figure 1 : m-bit comparator.

How to build 6-bit comparator?

Inputs:

A = A5A4A3A2A1A0

B = B5B4B3B2B1B0

12 bits in total → 4096 possible combinations

Not simple to design using conventional K-map techniques

The magnitude comparator can be designed at a higher level

Let us implement first the **EQ output** (A is equal to B)

1.EQ output

EQ = 1 ↔ A5 = B5 , A4 = B4, A3 = B3 , A2 = B2 , A1 = B1 , and A0 = B0

Define: Ei = AiBi + Ai′Bi′

Therefore, EQ = E5E4E3E2E1E0

2.The Greater Than Output

Given the 4-bit input numbers: A and B

1. If $A3 > B3$ then $GT = 1$, irrespective of the lower bits of A and B

Define: $G3 = A3B3'$ ($A3 = 1$ and $B3 = 0$)

2. If $A3 = B3$ ($E3 = 1$), we compare A2 with B2

Define: $G2 = A2B2'$ ($A2 = 1$ and $B2 = 0$)

3. If $A3 = B3$ and $A2 = B2$, we compare A1 with B1

Define: $G1 = A1B1'$ ($A1 = 1$ and $B1 = 0$)

4. If $A3 = B3$ and $A2 = B2$ and $A1 = B1$, we compare A0 with B0

Define: $G0 = A0B0'$ ($A0 = 1$ and $B0 = 0$)

Therefore, $GT = G5+E5G4+E5E4G3 + E5E4E3G2 + E5E4E3E2G1 + E5E4E3E2E1G0$.

3.The Less Than Output

Knowing GT and EQ, we can also derive $LT = (GT + EQ)'$ [2].

Design philosophy

As a Comparator have a delay time, we must have a two Flip Flop one for input and one for output, we use the first Flip flop as a Register to prevent a new input enter the comparator until previous inputs is calculation so that no mistake happens, and we use second Flip flop as Register to output the outputs at the same time in structurally comparator and Verification (Behaviorally) comparator to avoid the mistake.

In my project depended in my ID delay I see that maximum latency of the comparator is 30ns so that maximum frequency of the clock (33.3MHZ) ns so every rise of clock the new input will pass to comparator and output will show.

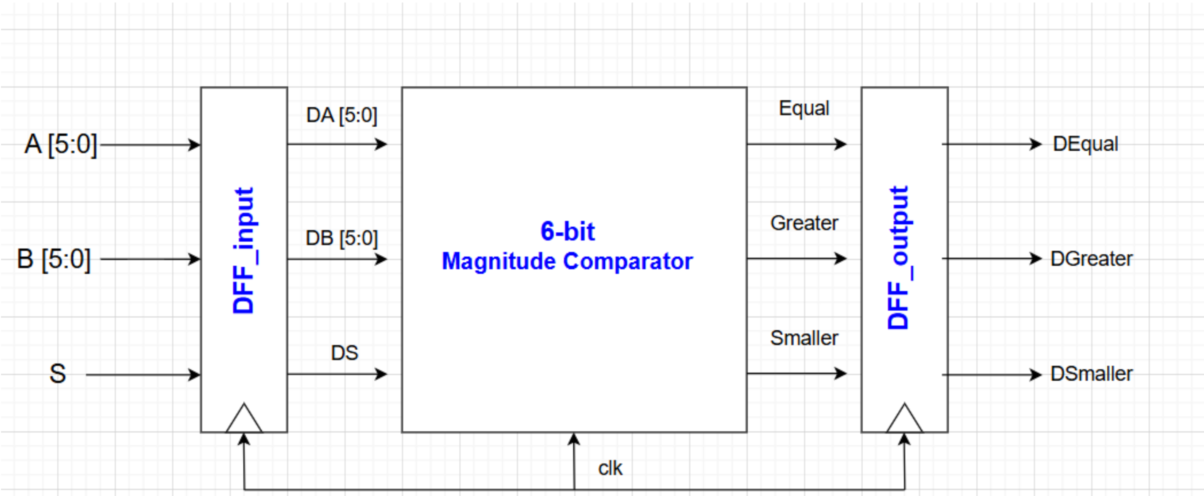

Figure 2 : project Design.

Simulation Results

Pass simulation →

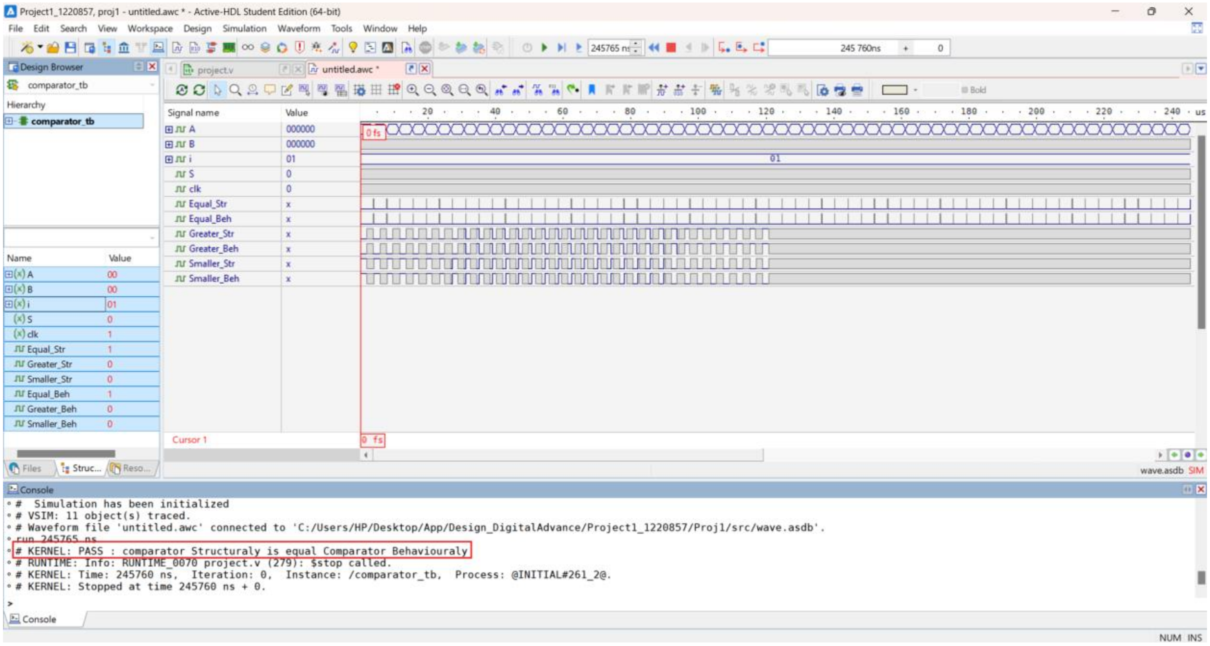

Figure 3 : Pass simulation.

Fail Simulation (We put or gate instead of and gate)→

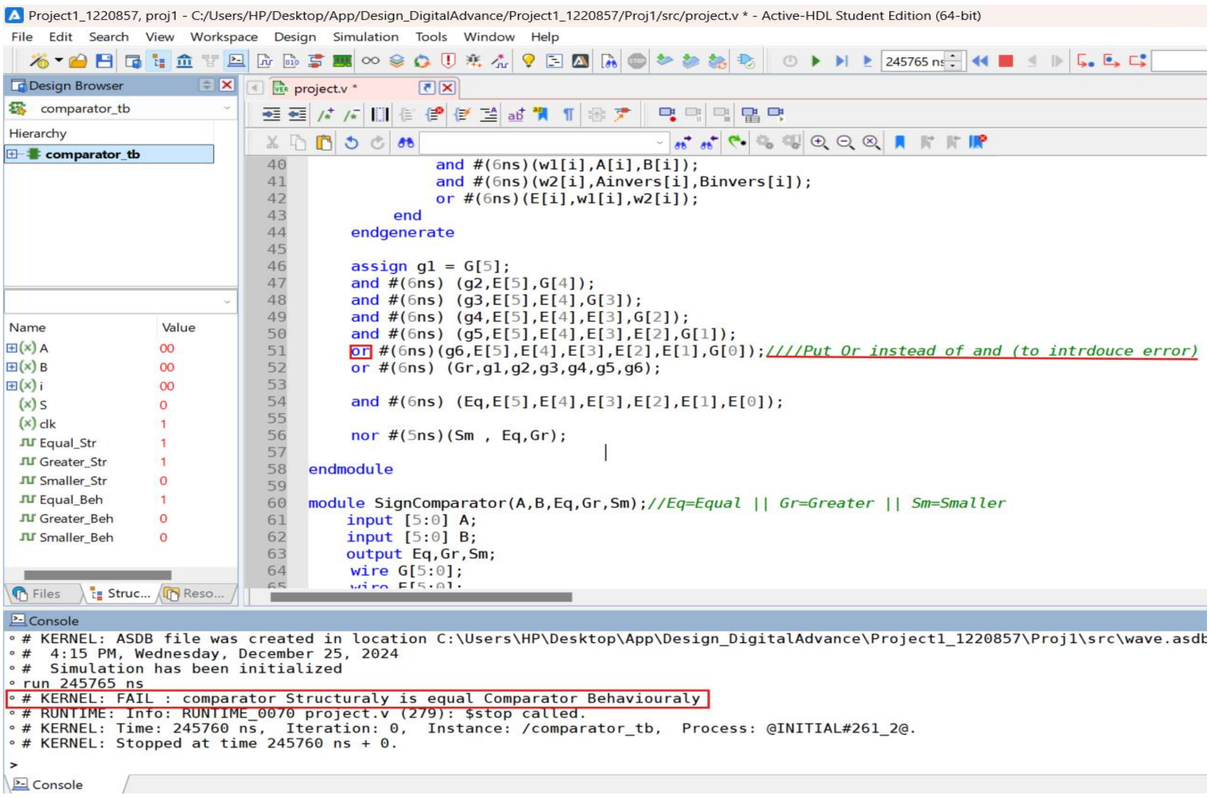

Figure 4 : Fail simulation by make error.

Fail Simulation (In this case I put the latency 15ns so the simulation is Fail)→

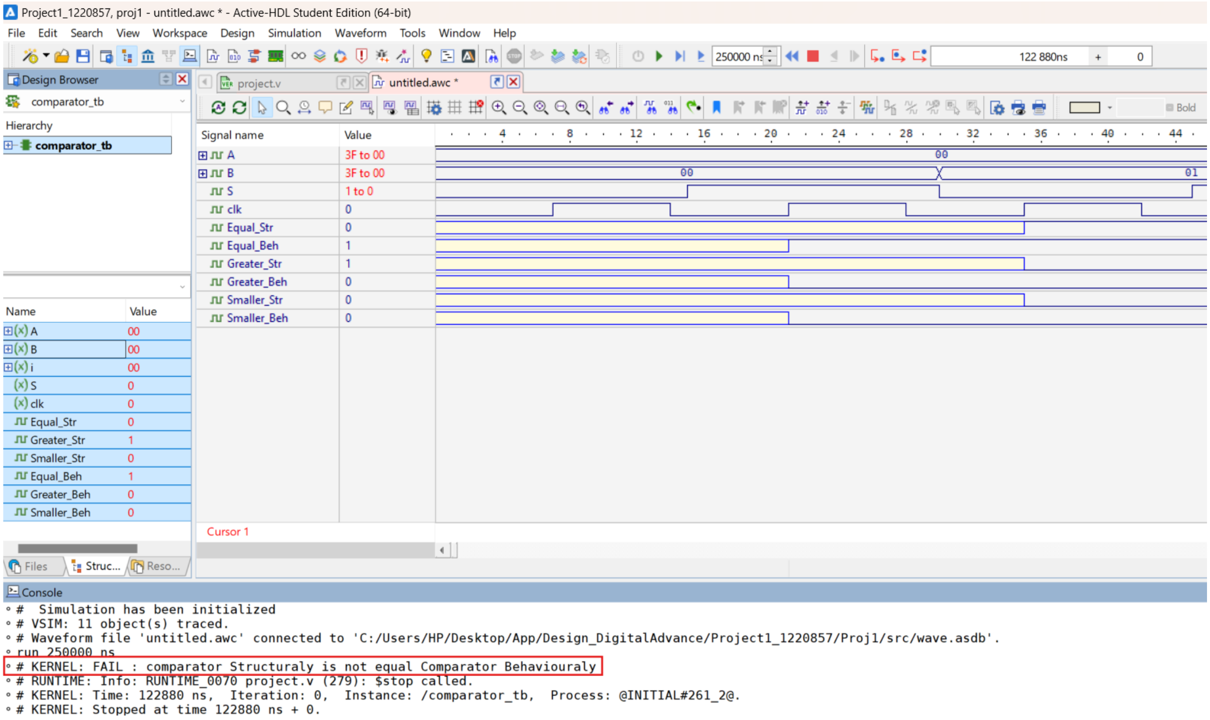

Figure 5 : simulation when change latency.

Samples

Case 1: A=B, S =0 → in first clock(15ns) the data will pass to comparator (A=0, B=0, S=0) in second clock(45ns) the result is shown and new inputs (A=0, B=0,S=1) will pass to comparator .

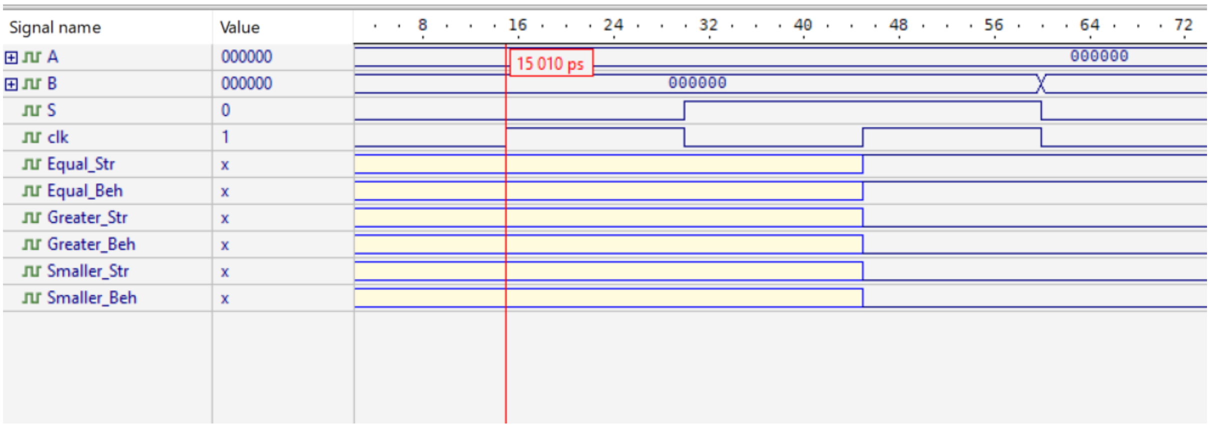

Figure 6 : case 1.

→Result of (A=0, B=0, S=0)

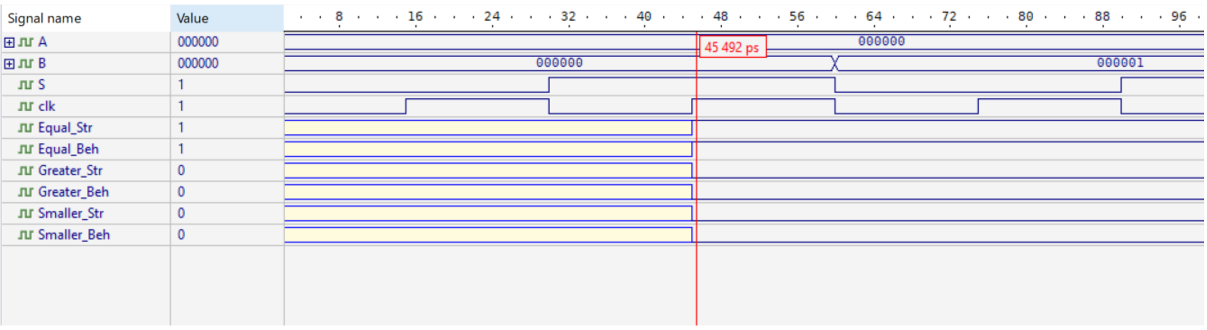

Figure 7 : case 1 Result.

Case 2: A=B, S =1 → in first clock(45ns) the data will pass to comparator (A=0, B=0, S=1) in second clock(75ns) the result is shown and new inputs will pass to comparator.

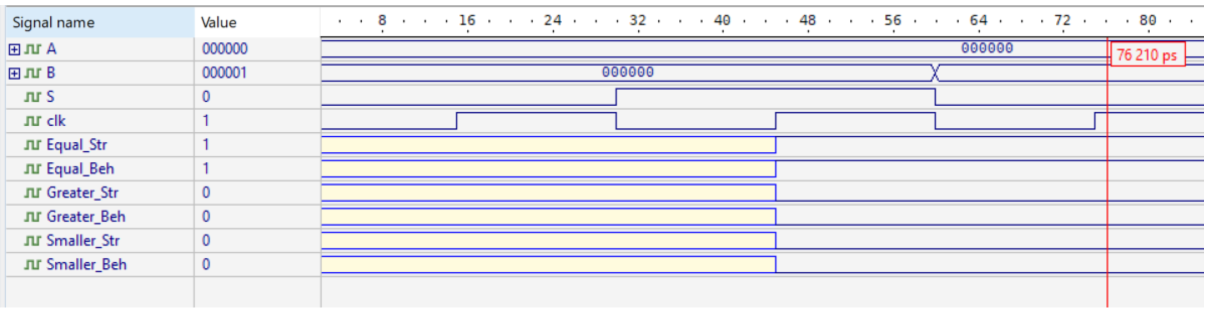

Figure 8 : case 2.

Case 3: A < B, S =0 → in first rise of clock(1935ns) the inputs will pass to comparator (A=0, B=100000, S=0) in second rise of clock(1965ns) the result is shown and new inputs (A=0, B=0, S=1) will pass to comparator.

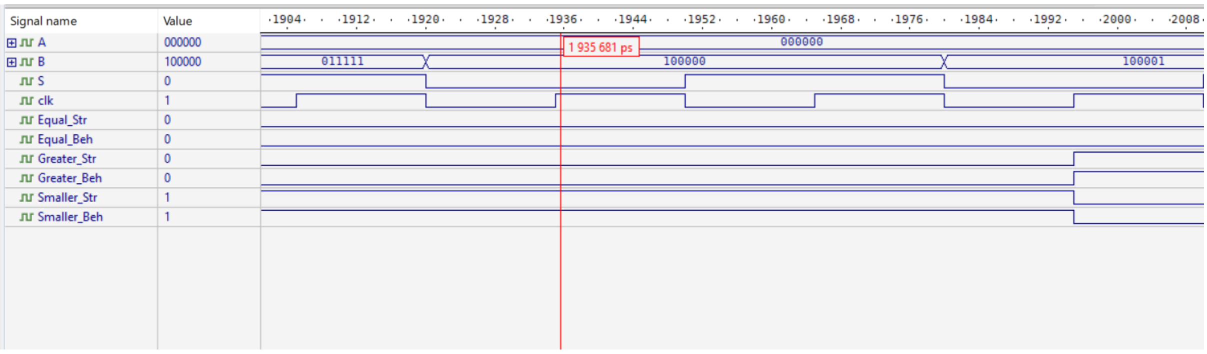

Figure 9 : case 3.

→Result of (A=0, B=100000, S=0)

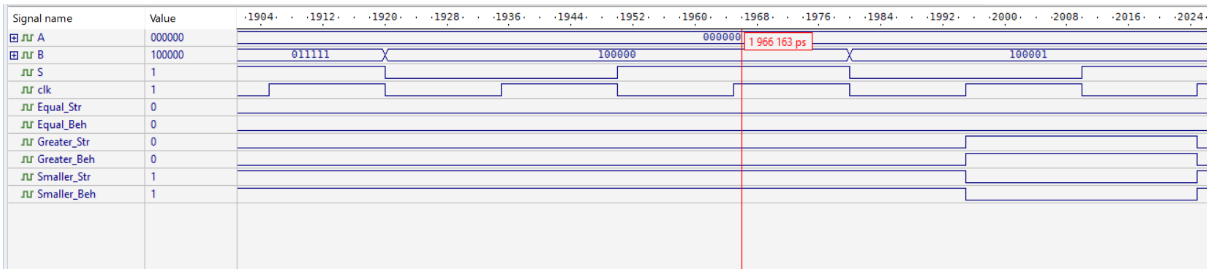

Figure**10** : case 3 Result.

Case 4: A < B, S =1 → in first rise of clock(1965ns) the inputs will pass to comparator (A=0, B=100000, S=1) in second rise of clock(1995ns) the result is shown and new inputs (A=0, B=0, S=1) will pass to comparator.

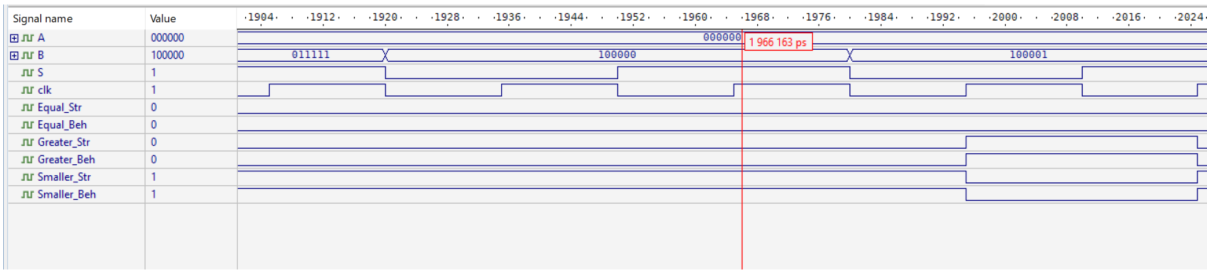

Figure 11 : case 4.

→Result of (A=0, B=100000, S=1)

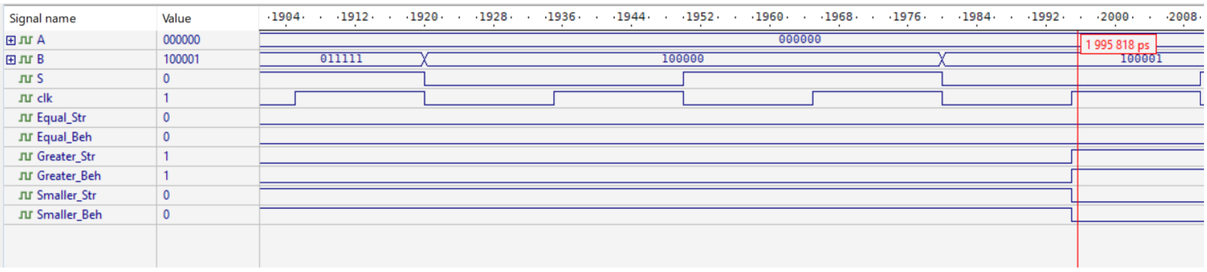

Figure 12 : case 4 Result.

Case 5: A > B, S =0 → in first rise of clock(149955ns) the inputs will pass to comparator (A=100111, B=000011, S=0) in second rise of clock(149985ns) the result is shown and new inputs will pass to comparator.

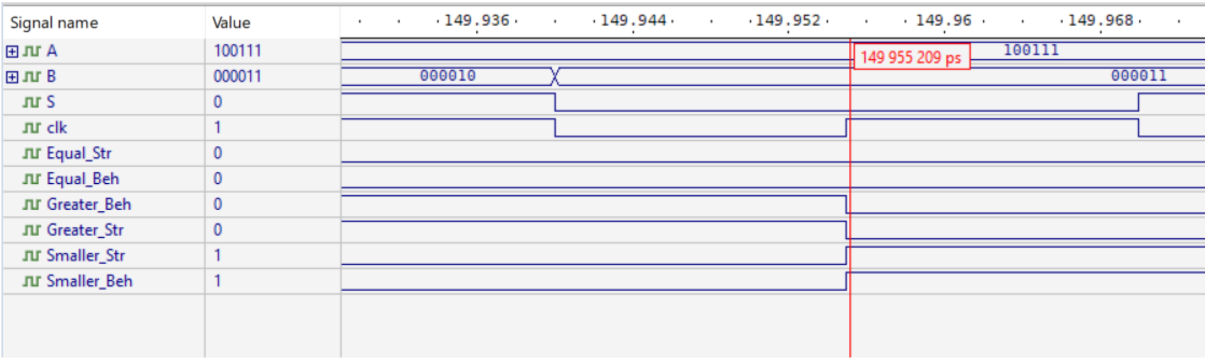

Figure 13 : case 5.

→Result of (A=100111, B=000011, S=0)

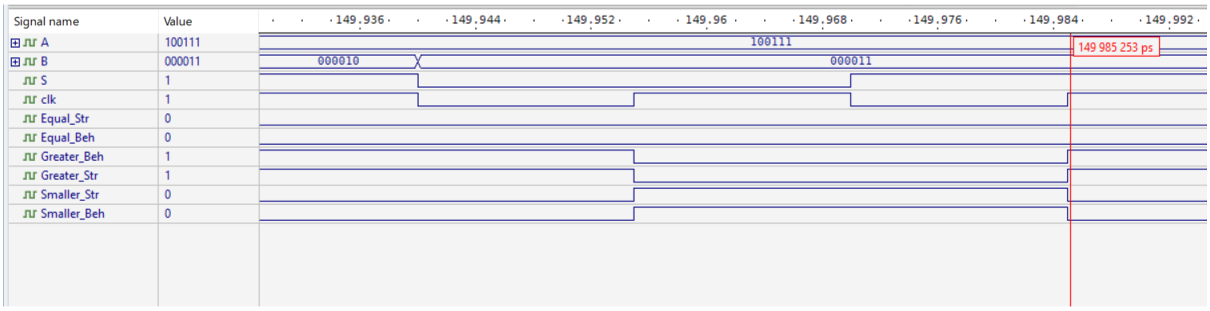

Figure 14 : case 5 Result.

Case 6: A > B, S =1 → in first rise of clock(147945ns) the inputs will pass to comparator (A=100110, B=100001, S=1) in second rise of clock(147975ns) the result is shown and new inputs will pass to comparator.

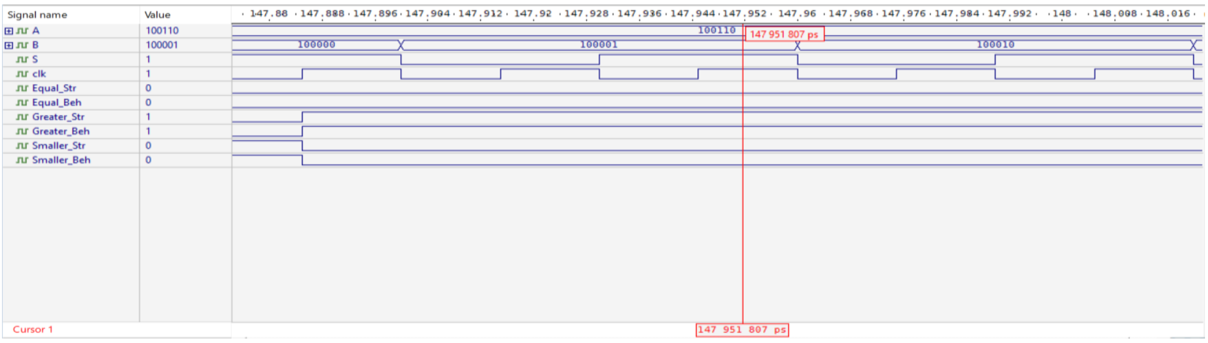

Figure 15 : case 6.

→Result of (A=100110, B=100001, S=1) => A = -26 , B = -31 (2'complenet) so A>B

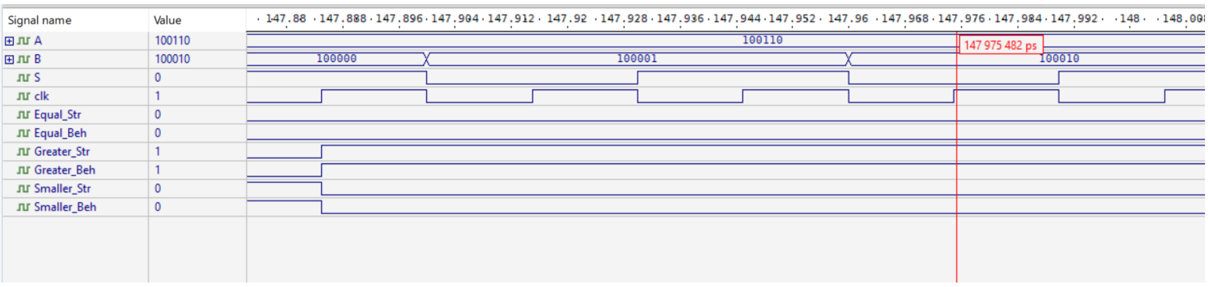

Figure 16 : case 6 Result.

Conclusion:

In conclusion, we design a structural comparator for both signed and unsigned numbers and see how comparisons between two numbers are performed logically, after we finish the project, we emphasizing the importance of structural design in digital systems. Through simulation and verification using Verilog in Active-HDL, we see the functionality and reliability of the comparator for all possible input combinations. We introduce an error to show that the verification is fail. We know the benefit of using Register when we use verification between two modules that it avoids making mistakes due to delay in structurally module.

Future work could involve building complex system harder than this to strengthen myself in Verilog.

References:

[1]: Digital Electronics - Comparators.
[2]: Ch4-+Combinational+Logic+Design.pdf - Google Drive